



University of
Applied Sciences

TECHNIKUM

WIEN

InterProcess communication (IPC)

IPC

- **Data exchange between processes**
 - data is safe in virtual address space
 - No other process is allowed to access another process'
- **But sometimes...**
 - more processes need to share data
 - processes depend on each other and need to synchronize their doing
 - output of one process needs to be handed over to another process
 - Coordinate the usage of system resources

IPC

- **Synchronization of processes**
 - **Timing of data manipulation**
 - Explicitly: synchronisation mechanisms
 - Implicitly: based on blocking data flow
- **Possibilities of IPC**
 - **Signals, (Un)named Pipes, Message Queues**
 - **Shared Memory, Semaphore**

Signals

- **Asynchronous IPC between processes**
 - e.g.: <Ctrl + C> in terminal (SIGINT)
 - e.g.: terminate a child process (SIGCHLD)
- **Sending of signals (signal.h)**
 - `int kill(pid_t pid,int signal)`

https://openbook.rheinwerk-verlag.de/c_von_a_bis_z/020_c_headerdateien_007.htm

Signals

- **Signalhandler**
 - routines for handling signals
 - Registered using `signal()`
 - Called **asynchronously** to normal flow when signal is called

```
1 void signal_handler(int sig) {  
2     // cleanup  
3 }  
4  
5 int main(int argc, char *argv[]) {  
6     // ...  
7     (void) signal(SIGHUP, signal_handler);  
8     (void) signal(SIGINT, signal_handler);  
9     (void) signal(SIGQUIT, signal_handler);  
10    // ...  
11 }
```

https://www.tutorialspoint.com/c_standard_library/c_function_signal.htm

Implicit Synchronization

- **Synchronization**
- **Automatically by using IPC mechanisms**
 - Send / Receive of messages
 - Writing in (named) pipes
 - Read in (named) pipes
- **No explicit synchronization mechanisms required**

Message Queues vs. Named Pipes

- **Message Queues**

- **Structured communication**
- **Exchange of messages via queues**
 - Messages == packet oriented
- **Communication between multiple sender / receiver possible**
- **Consuming receiver**

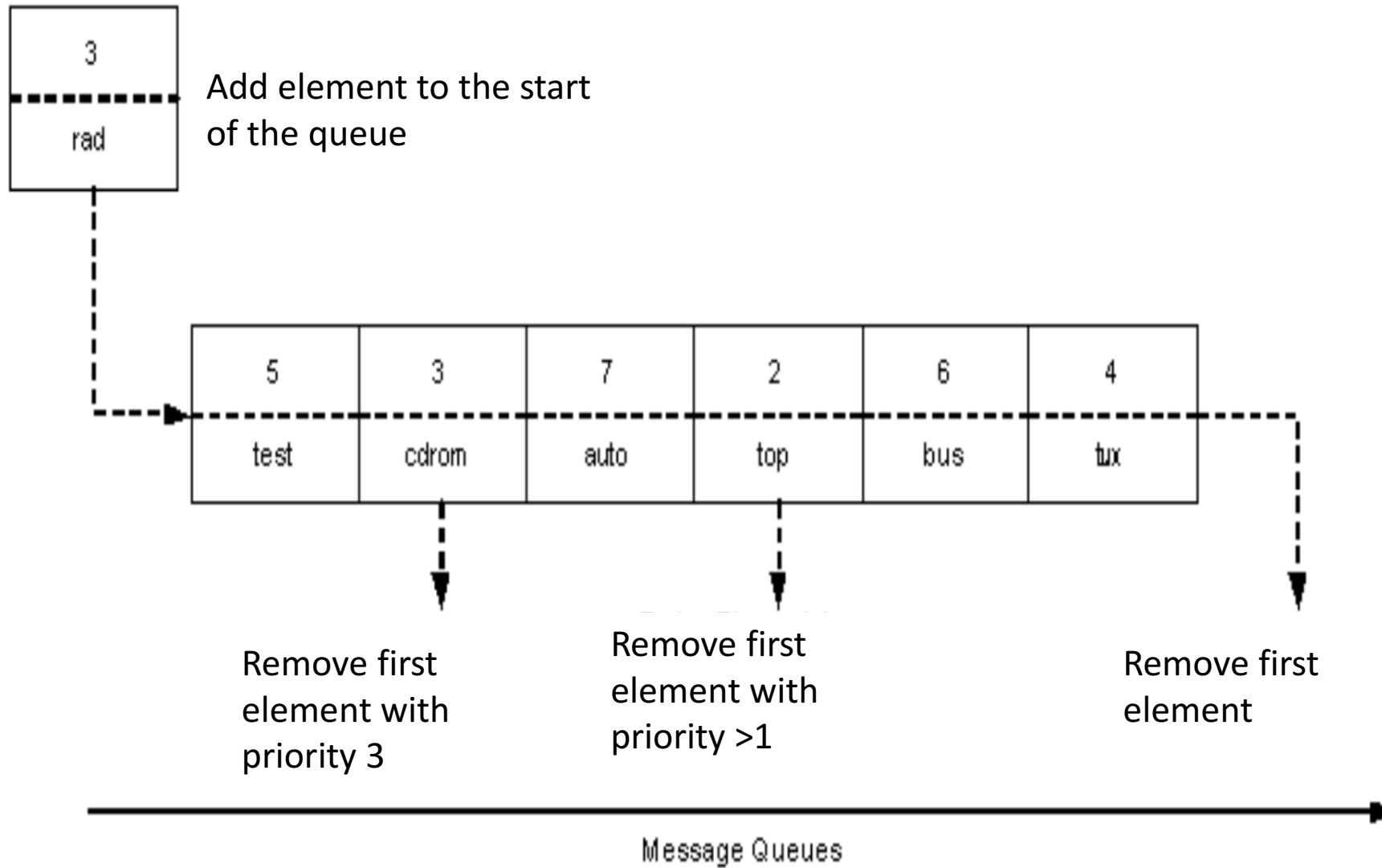
- **Named Pipe**

- **FIFO Data transmission (First In – First Out)**
- **Stream oriented**
 - Sequential reading only
- **Problems with multiple sender / receiver**

Message Queues

- **Queues for messages to a process**
 - each queue has a distinct identifier which is usable from other processes
 - **Works like a mailbox for a given process**
 - Process can push data into a message queue which can be fetched by other processes
- **Messages can be prioritized**
 - Fetch by FIFO or first message with defined priority

Message Queues



Message Queues

- **Operations**

- **Create Message Queue**
- **Send message**
- **Receive message**
- **Delete Message Queue**

- **Library functions in C available**

- **#include <sys/types.h>**
- **#include <sys/ipc.h>**
- **#include <sys/msg.h>**

Create Message Queue

- **int msgget (key_t key, int flags);**

- **Key**

- system-wide unique ID

- **Flags**

- Permissions
 - **Similar to file permissions (0660 => r/w for group and owner)**
- Creation Flags
 - **IPC_CREAT: creates queue if it does not exist**
 - **IPC_EXCL: returns an error if queue already exists**
 - **None: access to existing queue**
- Permission and Creation Flags are “or”-ed using a pipe (e.g.: 0660|IPC_CREAT|IPC_EXCL)

- **Returns queue-id or -1 in case of an error**

example

```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/msg.h>
4
5 // ...
6
7 #define KEY 9825007
8 #define PERM 0666
9
10 // ...
11
12 int msgid;
13
14 // ...
15
16 if( (msgid = msgget(KEY,PERM|IPC_CREAT|IPC_EXCL ))== -1 ){
17     /* error handling */
18 }
```

messaging

- **Self-defined message structure**

- first element must be a long and defines the message type
- the type (e.g.) defines the priority
- Further structure can freely be defined

```
1 typedef struct {  
2     long mType;  
3     char mText[MAX_DATA];  
4 } message_t;  
5  
6 message_t msg;
```

Send Message

- **int msgsnd (int msgid, const void *msg, size_t msgsize, int flags);**
 - **msgid**
 - Queue id (result of msgget())
 - **msg**
 - pointer to the message
 - **msgsize**
 - Size of payload (without the first long)
 - Typically: sizeof(msg) - sizeof(long)
 - **flags**
 - Send-Mode
 - IPC_NOWAIT: error if queue is full
 - Else: blocks if queue is full
- **Returns 0 or -1 on error (errno)**

Receive message

- **int msgrcv (int msgid, void *msg, size_t msgsize, long msgtype, int flags);**
 - **msgid**
 - Queue id (result of msgget())
 - **msg**
 - pointer to the message
 - **msgsize**
 - Size of payload (without the first long)
 - Typically: sizeof(msg) - sizeof(long)
 - **msgtype**
 - 0: first message
 - x: first message with type == x
 - -x: first message with type <= x
 - **flags**
 - Send-Mode
 - IPC_NOWAIT: error if queue is full
 - Else: blocks if queue is full
- **Returns number of bytes actually copied in the mText array or -1 on error (errno)**
- **Received message will be removed from queue**

Delete Message Queue

- **int msgctl (int msgid, int cmd, struct msqid_ds *buf);**
 - **msgid**
 - Queue id (result of msgget())
 - **cmd**
 - IPC_RMID: removes the queue
 - **buf**
 - Depends on cmd; IPC_RMID: NULL
- **Returns 0 or -1 on error (errno)**
- **Only 1 process should remove the queue.**

Message queue management from shell

- **ipcs -q**
 - Displays the message queues
- **ipcrm -q msgid**
 - Deletes the message queue based on the message id
- **ipcrm -Q key**
 - Deletes the message queue based on the message key

Named Pipes

- **Unstructured communication**
 - stream based
 - No structured data
 - usage similar to file access
 - FIFO Message flow
- **Consuming receiver**
 - Reading deletes message from pipe
- **Operations**
 - **Creation of a named pipe**
 - Also possible from the shell using mkfifo
 - **Deletion of a named pipe**
 - **Send and receive using stdio operations**

Named Pipes example

- **Often used in client / server applications**
 - **E.g.: Printer spooler**
 - **Client writes job in a named pipe**
 - **Server processes request (FIFO)**

Create Named Pipe

- **int mkfifo (const char *path, mode_t mode);**

- **Path**

- "filename"

- **Mode**

- Permission

- **Returns 0 or -1 on error (errno)**

```
1 if (mkfifo("foo", 0660) == -1) {  
2     /* error */  
3 }
```

Read / Write Named Pipe

- **Using stdio**
- **Open: fopen**
- **Read: fgets, fgetc**
- **Write: fputs, fputc, fprintf**
- **Close: fclose**

Remove Named Pipe

- **int remove (const char *path);**
 - **path**
 - “filename”
- **Returns 0 or -1 on error (errno)**
- **Only 1 process should remove the named pipe**

Named Pipe IPC

- **Open**

- A process using named pipes for reading is **blocked** until a process for writing is opened (vice versa).

- **Read**

- If a pipe is opened for writing by at least one process, reading blocks in case of an empty pipe
- If a pipe is not opened for writing, reading returns EOF (and does not block).

Named Pipe IPC

- **Write**

- If a pipe is opened for reading by at least one process, writing blocks in case of a full pipe
- If a pipe is not opened for reading, writing receives signal SIGPIPE (and does not block).

- **Multiple Reader/Writer**

- No low-level synchronization of reading and writing
- Chaos / data confusion / data jumble

Comparison

- **Message Queues**

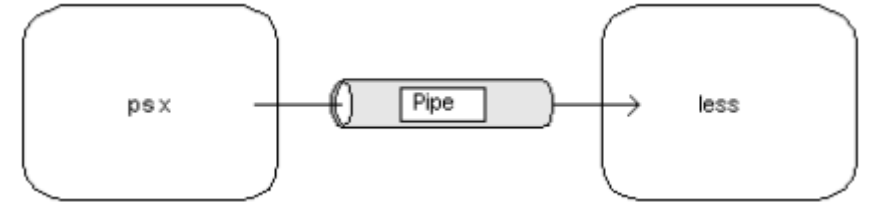
- message oriented
- Structured data
- Identified by numeric Key
- Message selection possible
- Read/write via API
- Special store in OS
- Concurrent writing into message queue is supported/managed by OS

- **Named Pipes**

- stream oriented
- Unstructured data
- Identified by filename
- Pure FIFO
- Read/write via stdio-functions
- Entry in filesystem
- Application needs to handle concurrency

Unnamed pipes

- **Communication between related processes**
- **Example: `ps x | less`**
 - Output from `ps x` is forwarded as input of `less`
- **Unidirectional connection (half-duplex)**
 - Data transmission only in 1 direction
 - Bidirectional communication achieved by 2 pipes
- **Synchronization between writing and reading process**
 - Write blocks in case of a full pipe
 - Read blocks in case of an empty pipe



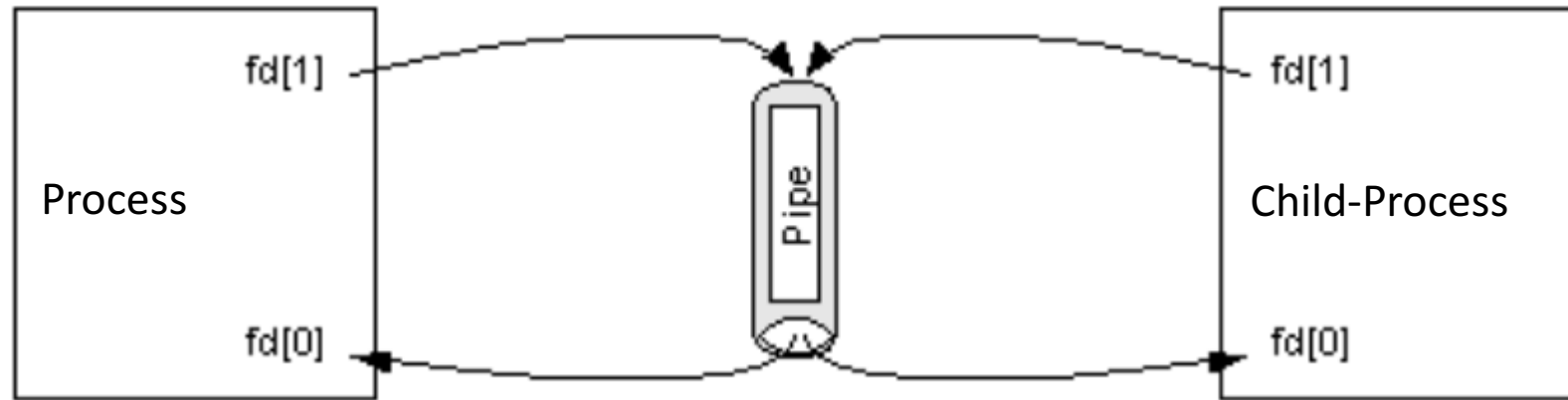
Create Pipes

- **Pipes declared by a field of 2 integer Elements**
 - **int fd[2];**
 - fd[0] ... for reading operations (compare stdin)
 - fd[1] ... for writing operations (compare stdout)
- **Open an unnamed pipe using system calls:**

```
1 #include <limits.h>
2 #include <unistd.h>
3
4 int fd[2];
5
6 // ...
7
8 if (pipe(fd)!=0) /* Error */
```

Pipes with fork()

- **The pipe is inherited into a child-process (fork())**



- **Afterwards the unused descriptor needs to be closed**
 - **Writing process closes read-descriptor (`close(fd[0]);`)**
 - **Reading process closes write-descriptor (`close(fd[1]);`)**
 - **Valid scenarios:**
 - Parent: reader, Child: writer
 - Parent: writer, Child: reader

Example

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <string.h>
6
7 int main(void) {
8     int fd[2], nbytes;
9     pid_t childpid;
10    char string[] = "Hello, world!\n";
11    char readbuffer[80];
12    pipe(fd);
13    if((childpid = fork()) == -1) {
14        perror("fork");
15        exit(1);
16    }
17
18    if(childpid == 0) {
19        /* Child process closes up input side of pipe */
20        close(fd[0]);
21
22        /* Send "string" through the output side of pipe */
23        write(fd[1], string, strlen(string)+1);
24        exit(0);
25    } else {
26        /* Parent process closes up output side of pipe */
27        close(fd[1]);
28
29        /* Read in a string from the pipe */
30        nbytes = read(fd[0], readbuffer, sizeof(readbuffer));
31        printf("Received string: %s", readbuffer);
32    }
33
34    return(0);
35 } // end of main
```

Further usage of unnamed pipe

- **After the creation of the pipe the standard i/o functions can be used for further access.**
- **Therefore, the file descriptor needs to be converted into a file pointer (using fdopen with file descriptor and mode).**

```
1 FILE *rd, *wr;  
2  
3 rd = fdopen(fd[0], "r");  
4 wr = fdopen(fd[1], "w");
```

Redirection of descriptors

- In case a child process using `exec()` calls an existing program the file descriptors for `stdin (0)` and `stdout (1)` can be redirected to a pipe using `dup2()`.
- `dup2 (int oldd, int newd)`
 - The old descriptor will be duplicated and connected to the new descriptor.

Example: connect stdin of a process to a pipe

```
1 #include <unistd.h>
2 #include <sys/types.h>
3 #include <fcntl.h>
4
5 if (dup2 (fd[0], STDIN_FILENO) != STDIN_FILENO) {
6     /* error using dup2 ... */
7 }
8
9 close (fd[0]); /* not required anymore */
```

popen

- **Merge of: pipe(), fork(), exec()**
 - Opens a pipe, create a child-process, closes the not required descriptors and overloads the child-process with an exec function
- **FILE *popen (char *command, char *type);**
 - **Command**
 - Filename of a program
 - **Type**
 - Pipe direction
 - „w“: written data is redirected to stdin of the command
 - „r“: output of the command is redirected to be available using the pipe
- **Use pclose() to close the opened file pointer**
 - pclose waits for the command to finish

example

```
1#include <stdio.h>
2#define MAX 256
3
4int main(int argc, char* argv[]) {
5    FILE *fp;
6    char puffer[MAX];
7
8    if ((fp=popen("ls -l", "r"))!=NULL) {
9        while (fgets(puffer, MAX, fp) != NULL) { /* ... */}
10    }
11
12    pclose (fp);
13    return 0;
14 }
```